Where Supabase fits well

- **Postgres over MongoDB**: For an HMS, this is actually a good direction. Healthcare data (patients, encounters, departments, staff, shifts, orders, lab results, billing) is inherently relational — lots of foreign keys, joins, referential integrity. Postgres will likely serve this domain better than MongoDB long-term.
- **Auth + Realtime**: Built-in auth and realtime subscriptions could genuinely simplify things like live bed/ward status, shift dashboards, or department queues.
- **Row Level Security (RLS)**: Postgres RLS is powerful for department-level data isolation (e.g., pharmacy can't see psychiatry records unless explicitly granted).

Where it gets complicated

1. Your Django backend isn't a drop-in replacement.
Supabase auto-generates REST/GraphQL APIs directly from your Postgres schema (via PostgREST) — it's not a general-purpose application server like Django. Whatever business logic currently lives in Django (validation rules, department workflows, billing calculations, integrations with lab/pharmacy systems, complex multi-step transactions) would need to move to:

- Postgres functions/triggers (SQL/PLpgSQL) — a real paradigm shift for a Python team, or
- Supabase Edge Functions (written in Deno/TypeScript, not Python) — meaning you'd rewrite business logic in a new language, or
- Keep Django running alongside Supabase's Postgres as a hybrid — which reduces the benefit of migrating and adds two things touching the same DB

For a 20-department HMS, there's likely substantial business logic. This is probably your biggest cost, not the DB swap.

2. Dynamic RBAC based on roster/shift needs custom design.
Supabase's default auth model assumes a fairly static user→role relationship via JWT claims. Your "role changes based on shift" requirement isn't out-of-the-box — you'd build it yourself with a `role_assignments` table joined into RLS policies (checking current shift against a schedule table). Very doable in Postgres, but it's engineering work, not a checkbox.

3. MongoDB → Postgres is a real schema migration.
Document structures don't map 1:1 to relational tables. You'd need proper schema design and data migration scripts — not just an export/import.

4. Compliance, if this handles PHI.
Supabase's hosted platform is HIPAA compliant, and PHI can be stored once you enter into a Business Associate Agreement and fulfill your obligations under a shared responsibility model. However, the HIPAA add-on and additional compliance controls for ePHI are available as an add-on, not by default on all plans, and a signed BAA plus the HIPAA add-on is required specifically when handling PHI. If self-hosting Supabase instead of using their managed cloud, the necessary HIPAA controls are not supported out of the box in self-hosted deployments — you'd own that compliance burden entirely. If this is for India rather than US patients, HIPAA may not even be the relevant framework (DPDP Act would be) — worth clarifying your regulatory jurisdiction. Supabase + 3

My honest take

This is less "migrate to Supabase" and more "re-platform the backend onto Postgres, potentially replacing Django with a mix of Postgres functions/RLS and Edge Functions." That's a legitimate architecture for a new HMS, but as a *migration* of an existing 20-department system, the effort is substantial and the risk is concentrated in re-implementing business logic, not in the database swap itself.

A **middle path** many teams choose: migrate MongoDB → Postgres, but keep Django as your backend layer (using Postgres directly via Django ORM), and adopt Supabase selectively for things it's genuinely great at — Auth, Realtime, Storage — without going all-in on PostgREST replacing your API layer. That gets you the relational data model benefits without a full rewrite of business logic.

Supabase tends to shine in a specific sweet spot. Here's where it fits best — and where it doesn't.

Best fit

1. Startups and MVPs that need to move fast
Auto-generated APIs, built-in auth, and instant Postgres mean a small team can go from idea to working app in days rather than weeks, without hand-building a backend.
2. Apps with straightforward CRUD + relational data
SaaS tools, internal dashboards, marketplaces, booking systems, CRMs — anything where most operations are "read/write records with relationships" rather than heavy custom business logic.
3. Realtime-heavy applications
Chat apps, live collaboration tools, dashboards that need to update instantly (live order status, presence indicators, multiplayer features) — Supabase's Realtime subscriptions are a genuine strength here.
4. Apps needing file storage alongside structured data
Content platforms, portfolio sites, apps with user uploads (images, documents, videos) where you want storage and database access unified.
5. AI/RAG applications
With pgvector built in, Supabase is a popular choice for AI apps needing vector embeddings alongside regular relational data — e.g., a support chatbot that also needs user accounts and structured records.
6. Mobile and web apps needing quick auth
Social logins, magic links, JWT-based auth out of the box — useful for consumer apps that don't want to build auth from scratch.
7. Indie developers / small teams / side projects
Generous free tier, minimal ops overhead, and a good developer experience make it popular for solo builders and small startups.

Where it's a weaker fit

- **Apps with heavy, complex business logic** — if most of your value is in custom server-side workflows, validations, and orchestration (not just data access), you'll end up writing a lot of that logic in Postgres functions or Edge Functions (Deno/TypeScript), which is a different paradigm than a traditional app server like Django, Rails, or Express.
- **Highly regulated, compliance-heavy systems** (healthcare, finance) — usable, but requires deliberate configuration (BAAs, HIPAA add-ons, self-hosting decisions) rather than being compliant by default.
- **Systems needing a different database engine** — if your data genuinely is document-shaped or you need a specialized DB (graph, time-series at scale), forcing it into Postgres isn't ideal.
- **Very large enterprise systems with complex microservices** — Supabase is more of a "backend-in-a-box" for a single Postgres instance; sprawling microservice architectures with multiple data stores don't map cleanly onto it.

Rule of thumb: if your app is mostly "store, query, and react to relational data with auth," Supabase is an excellent fit. If your app is mostly "orchestrate complex business processes," Supabase can still store your data well, but you'll be writing meaningful backend logic yourself either way — the question is just where that logic lives.